



DEPARTAMENTO
DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Representación de la información

Números fraccionarios

Sistemas Digitales

2do Cuatrimestre 2026

FCEN - UBA

Punto flotante (IEEE 754)



IEEE: Instituto de Ingenieros Eléctricos y Electrónicos

Estándar IEEE 754

- ▶ Establecido en 1985
 - IEEE 754-1985 - IEEE Standard for Binary Floating-Point Arithmetic
 - IEEE 754-2008 - IEEE Standard for Floating-Point Arithmetic
 - última revisión: IEEE 754-2019 - IEEE Standard for Floating-Point Arithmetic
- ▶ Soporte general
 - Hardware
 - Software
- ▶ Desarrollado *mirando* las necesidades numéricas
 - Formas estándar de redondeo, *overflow*, *underflow*
 - Difícil obtener realizaciones veloces en hardware
 - Análisis numérico vs diseño de hardware

Expresión numérica

$$f = (-1)^s M 2^E$$

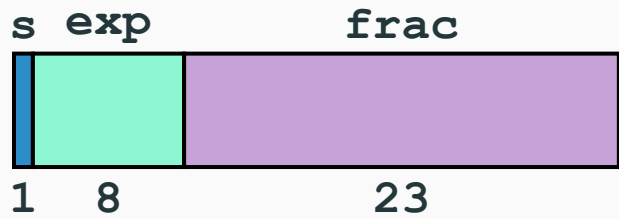
- ▶ **s**: bit de signo
- ▶ **M**: significando (normalmente, en el intervalo $[1.0, 2.0)$)
- ▶ **E**: exponente, pesa el valor por una potencia de 2

Codificación (almacenamiento)

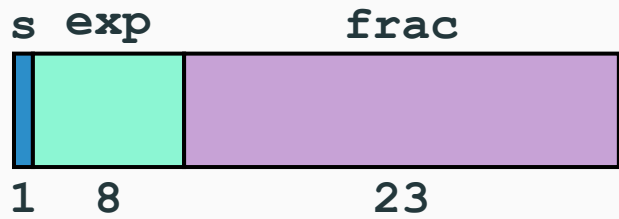
- ▶ El msb **s** es el bit de signo **s**
- ▶ **exp** codifica **E**, pero no es **E**
- ▶ **frac** codifica **M**, pero no es **M**



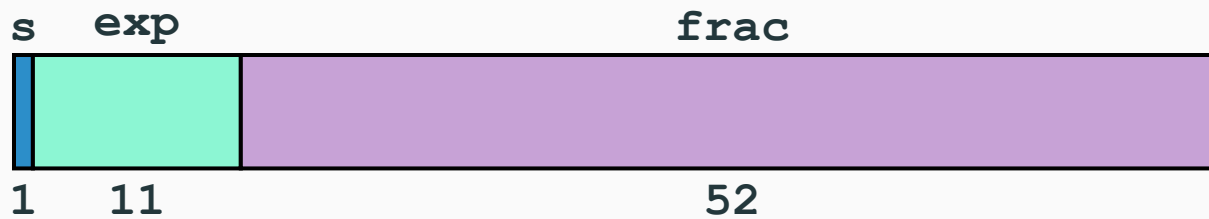
Precisión simple: 32 bits



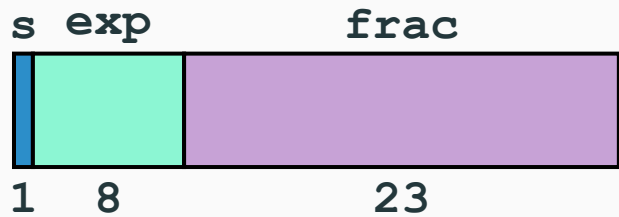
Precisión simple: 32 bits



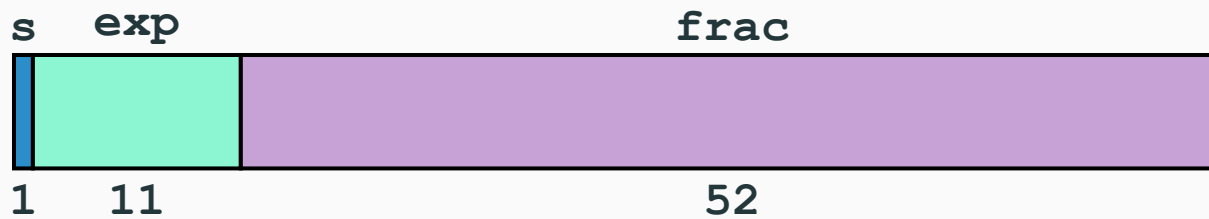
Precisión doble: 64 bits



Precisión simple: 32 bits



Precisión doble: 64 bits



Precisión extendida: 80 bits (Intel)



$$v = (-1)^s M 2^E$$

- ▶ Cuando **exp** $\neq$ 000 ... 0 y **exp** $\neq$ 111 ... 1
- ▶ Exponente codificado por exceso: $E = \mathbf{Exp} - bias$
 - **Exp**: valor **unsigned** del campo **exp**
 - *bias*: $2^{k-1} - 1$, donde k es el número de bits del campo **exp**
 - Simple: 127 (**Exp**: 1, ..., 254; E : -126, ..., 127)
 - Doble: 1023 (**Exp**: 1, ..., 2046; E : -1022, ..., 1023)
- ▶ Significando codificado con parte entera implícita en 1
$$M = 1.sss \dots s_2$$
 - *sss ... s*: bits del campo **frac**
 - Mínimo cuando **frac** = 00 ... 0 ($M = 1.0$)
 - Máximo cuando **frac** = 11 ... 1 ($M = 2.0 - \epsilon$)

Ejemplo de codificación normalizada

Valor: 49374.0;

$$49374.0_{10} = 1100000011011110_2 \\ 1.100000011011110_2 \times 2^{15}$$

$$v = (-1)^s M 2^E$$

$$E = \mathbf{Exp} - bias$$

Significando

$$\mathbf{M} = 1.100000011011110_2$$

$$\mathbf{frac} = 100\ 0000\ 1101\ 1110\ 0000\ 0000_2$$

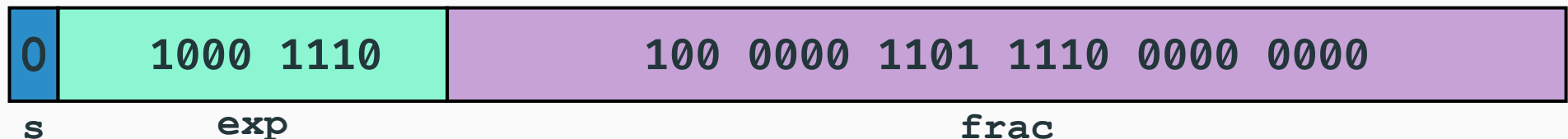
Exponente

$$\mathbf{E} = 15$$

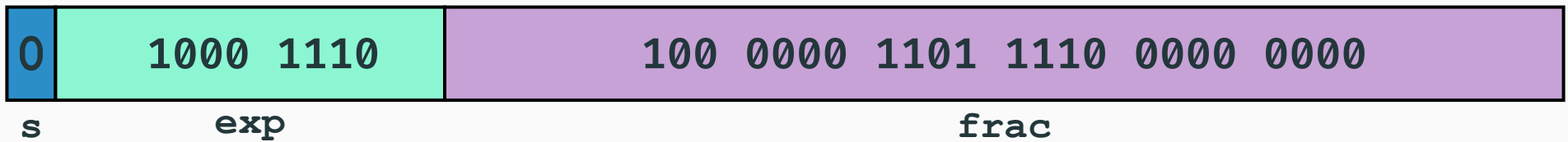
$$bias = 127$$

$$\mathbf{exp} = 142 = 10001110$$

Resultado



Ejemplo de codificación normalizada



Hexa: 4 7 4 0 D E 0 0

Binario: 0100 0111 0100 0000 1101 1110 0000 0000

142: 100 0111 0

49374: 1100 0000 1101 1110

Condición: $\mathbf{exp} = 000 \dots 0$

► Exponente: $E = 1 - bias$ (en vez de $0 - bias$)

- Simple: -126
- Doble: -1022

► Significando codificado con parte entera implícita en 0:

$M = 0.sss \dots s_2$

► $sss \dots s$: bits del campo **frac**

► Casos

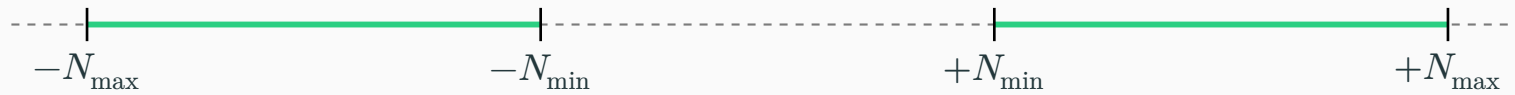
- $\mathbf{exp} = 000 \dots 0, \mathbf{frac} = 000 \dots 0$
 - Representa el valor cero (0)
 - Distintos valores para +0 y -0
- $\mathbf{exp} = 000 \dots 0, \mathbf{frac} \neq 000 \dots 0$
 - Valores más cercanos a 0.0
 - Equiespaciados

$$v = (-1)^s M 2^E$$
$$E = 1 - bias$$

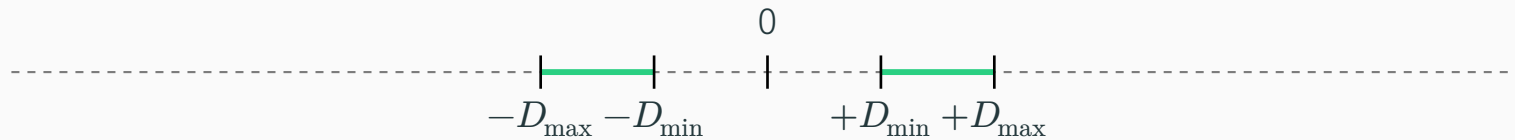
Condición: $\mathbf{exp} = 111 \dots 1$

- ▶ Caso: $\mathbf{exp} = 111 \dots 1$, $\mathbf{frac} = 000 \dots 0$
 - Representa el valor ∞ (infinito)
 - Operaciones que dan *overflow*
 - Hay positivo y negativo
 - Ej.: $1.0/0.0 = -1.0/-0.0 = +\infty$, $1.0/-0.0 = -\infty$
- ▶ Caso: $\mathbf{exp} = 111 \dots 1$, $\mathbf{frac} \neq 000 \dots 0$
 - NaN: Not-A-Number
 - Representa casos donde no se puede determinar un valor numérico
 - Ej.: $\sqrt{-1}$, $\infty - \infty$, $\infty \times 0$

No todos los reales del rango son representables



Números normalizados



Números denormalizados

$$\begin{array}{ll} N_{\max} & 3.4 \times 10^{38} \\ N_{\min} & 1.18 \times 10^{-38} \end{array}$$

$$\begin{array}{ll} D_{\max} & 1.18 \times 10^{-38} \\ D_{\min} & 1.4 \times 10^{-45} \end{array}$$

Normalizados

$N_{\min}$

0x0080 0000
0 00000001 000000000000000000000000
exp = 1, frac = 0
 $N_{\min} = 1.0_2 \times 2^{-126}$
($\approx 1.18 \times 10^{-38}$)

$N_{\max}$

0x7F7F FFFF
0 11111110 111111111111111111111111
exp = 254, frac =
1111111111111111111111111B
 $N_{\max} = 1.11...1_2 \times 2^{127} = (2 - 2^{-23}) \times 2^{127}$
($\approx 3.4 \times 10^{38}$)

Denormalizados

$D_{\min}$

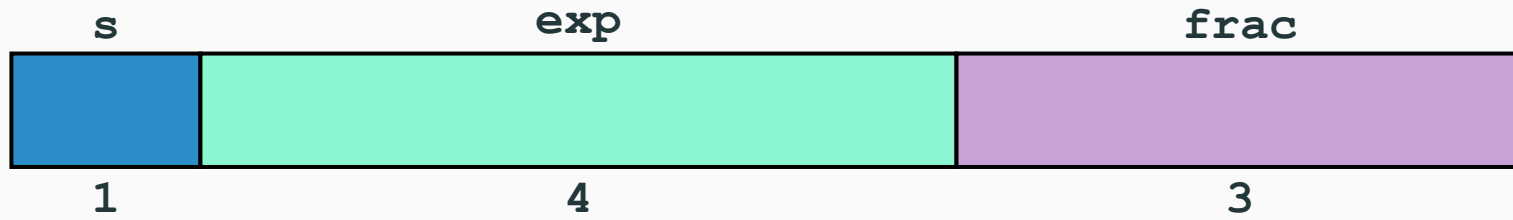
0x0000 0001
0 00000000 000000000000000000000001
exp = 0, frac = 0000000000000000000000001B
 $D_{\min} = 0.0...01_2 \times 2^{-126} = 2^{-23} \times 2^{-126} = 2^{-149}$
($\approx 1.4 \times 10^{-45}$)

$D_{\max}$

0x007F FFFF
0 00000000 111111111111111111111111
exp = 0, frac = 1111111111111111111111111B
 $D_{\max} = 0.1...1_2 \times 2^{-126} = (1 - 2^{-23}) \times 2^{-126}$
($\approx 1.18 \times 10^{-38}$)

Ejemplos y propiedades

Ejemplo con un float pequeño



Representación en punto flotante de 8 bits

- el bit de signo es el msb
- el exponente es de 4 bits: $bias = 7$
- la parte fraccionaria: 3 bits

Misma idea que el formato IEEE:

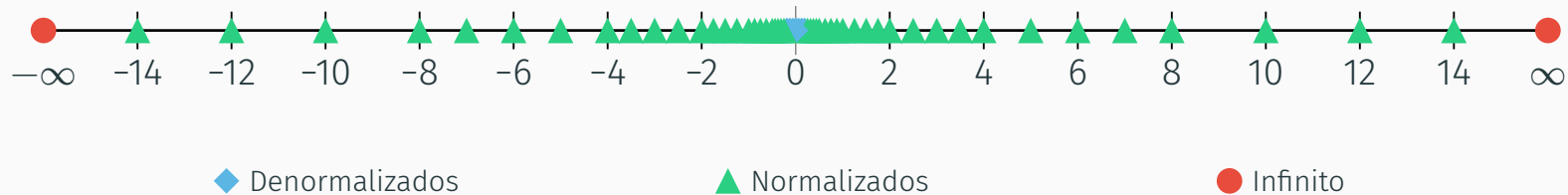
- normalizados, denormalizados
- representación del 0, NaN, infinito

Rango Dinámico (positivos)

	s	exp	frac	E	Valor	Nota
Números denormalizados	0	0000	000	-6	$0 \times 2^{-6} = 0$	cero
	0	0000	001	-6	$1/8 \times 2^{-6} = 1/512$	denormalizado menor
	0	0000	010	-6	$1/4 \times 2^{-6} = 1/256$	
		:	:	:	:	
	0	0000	110	-6	$3/4 \times 2^{-6} = 3/256$	
	0	0000	111	-6	$7/8 \times 2^{-6} = 7/512$	denormalizado mayor
Números Normalizados	0	0001	000	-6	$1 \times 2^{-6} = 1/64$	normalizado menor
	0	0001	001	-6	$9/8 \times 2^{-6} = 9/512$	más cercano a 1 (menor)
		:	:	:	:	
	0	0110	110	-1	$7/4 \times 2^{-1} = 7/8$	
	0	0110	111	-1	$15/8 \times 2^{-1} = 15/16$	
	0	0111	000	0	$1 \times 2^0 = 1$	1
	0	0111	001	0	$9/8 \times 2^0 = 9/8$	más cercano a 1 (mayor)
	0	0111	010	0	$5/4 \times 2^0 = 5/4$	
		:	:	:	:	
	0	1110	110	7	$7/4 \times 2^7 = 224$	
	0	1110	111	7	$15/8 \times 2^7 = 240$	normalizado mayor
	0	1111	000	-	∞	infinito

float tipo IEEE de 6 bits

- fracción de 2 bits
- exponente de 3 bits
- $bias = 2^{3-1} - 1 = 3$



float tipo IEEE de 6 bits

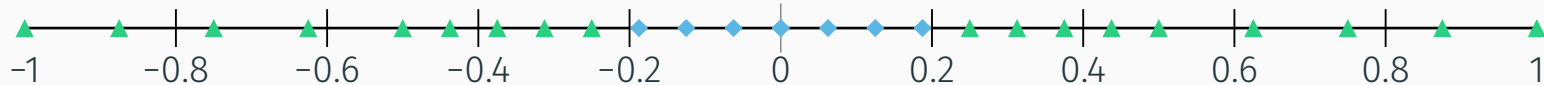
- fracción de 2 bits
- exponente de 3 bits
- $bias = 2^{3-1} - 1 = 3$



Negativos



Cerca de 0



Positivos



El cero de punto flotante (FP) es el mismo que con los enteros

- ▶ Todos los bits en 0

Casi sirve la comparación de enteros

- ▶ Comparar primero los bits de signo
- ▶ Tener en cuenta que FP tiene 2 ceros (-0 y $+0$)
- ▶ NaN es problemático
 - Es mayor que cualquier otro número
 - ¿Qué debe dar la comparación?
- ▶ Caso contrario, está bien
 - Denormalizados vs Normalizados
 - Normalizados vs Infinito

Descripción	exp	frac	Valor numérico
Cero	00 ... 00	00 ... 00	0.0
Denormalizado menor	00 ... 00	00 ... 01	$2^{-\{23,52\}} \times 2^{-\{126,1022\}}$ simple $\approx 1.4 \times 10^{-45}$ doble $\approx 4.94 \times 10^{-324}$
Denormalizado mayor	00 ... 00	11 ... 11	$(1.0 - \varepsilon) \times 2^{-\{126,1022\}}$ simple $\approx 1.18 \times 10^{-38}$ doble $\approx 2.23 \times 10^{-308}$
Normalizado menor	00 ... 01	00 ... 00	$1.0 \times 2^{-\{126,1022\}}$
Uno (1)	01 ... 11	00 ... 00	1.0
Normalizado mayor	11 ... 10	11 ... 11	$(2.0 - \varepsilon) \times 2^{\{127,1023\}}$

Redondeo, suma y multiplicación

- ▶ $u +_f v = \text{Round}(u + v)$
- ▶ $u \times_f v = \text{Round}(u \times v)$

Idea básica

- ▶ Primero obtener el resultado **exacto**
- ▶ Ajustarlo a la precisión disponible
 - Puede haber overflow si el exponente es grande
 - **Redondear** para que entre en **frac**

- ▶ Hay 4 modos de redondeo
 - Hacia 0, $-\infty$, $+\infty$, el más cercano (*nearest*)

Ejemplo

Hacia	\$1.40	\$1.60	\$1.50	\$2.50	-\$1.50
0	\$1.00	\$1.00	\$1.00	\$2.00	-\$1.00
abajo ($-\infty$)	\$1.00	\$1.00	\$1.00	\$2.00	-\$2.00
arriba ($+\infty$)	\$2.00	\$2.00	\$2.00	\$3.00	-\$1.00
el más cercano	\$1.00	\$2.00	\$2.00	\$2.00	-\$2.00

El redondeo por omisión es al (par) más cercano, llamado
Round-to-Even

- ▶ Es el modo por omisión
 - No es simple configurar otro modo
 - Todos los demás están estadísticamente sesgados

Ejemplo: redondeando a 2 decimales

- ▶ Cuando se está justo en el medio entre 2 valores
 - Redondear para que el dígito menos significativo sea par

Ejemplo

7.8949999	7.89	(Por debajo de la mitad)
7.8950001	7.90	(Por arriba de la mitad)
7.8950000	7.90	(Justo en la mitad – redondear hacia arriba)
7.8850000	7.88	(Justo en la mitad – redondear hacia abajo)

- ▶ Números fraccionarios
 - “Par” cuando el bit menos significativo es 0
 - “En la mitad” cuando los bits a la derecha de la posición de redondeo son: $100 \dots 0_2$

Ejemplo

- ▶ Redondear al $1/4$ más cercano (2 bits después del punto)

Valor	Binario	Redondeado	Acción	Final
$2 \frac{3}{32}$	10.00 011	10.00	$<1/2$ – abajo	2
$2 \frac{3}{16}$	10.00 110	10.01	$>1/2$ – arriba	$2 \frac{1}{4}$
$2 \frac{7}{8}$	10.11 100	11.00	$1/2$ – arriba	3
$2 \frac{5}{8}$	10.10 100	10.10	$1/2$ – abajo	$2 \frac{1}{2}$

Operación: $(-1)^{s_1} M_1 2^{E_1} \cdot (-1)^{s_2} M_2 2^{E_2}$

Resultado Exacto: $(-1)^s M 2^E$

signo $S \ s_1 \oplus s_2$
significando $M \ M_1 \cdot M_2$
exponente $E \ E_1 + E_2$

Ajuste

- ▶ Si $M \geq 2$, desplazar M a la derecha, incrementar E
- ▶ Si E fuera de rango, *overflow*
- ▶ Redondear M para que concuerde con la precisión de **frac**

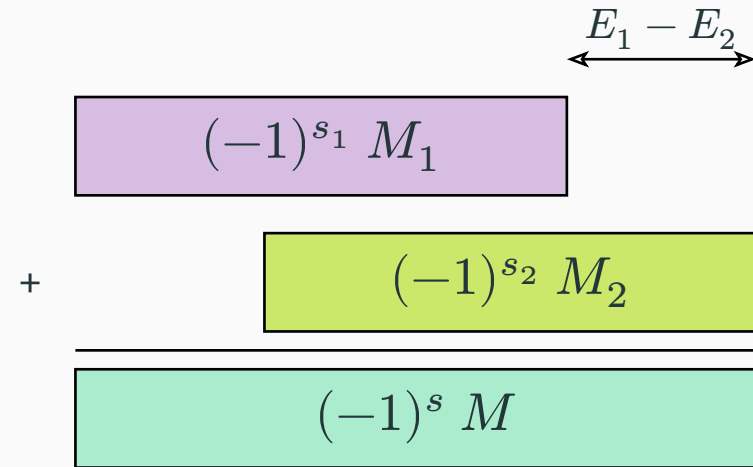
Implementación

- ▶ Lo más difícil es multiplicar los significandos

Operación: $(-1)^{s_1} M_1 2^{E_1} + (-1)^{s_2} M_2 2^{E_2}$

Resultado Exacto: $(-1)^s M 2^E$,

- ▶ signo s , significando M :
 - Resultado de alinear y sumar
- ▶ exponente E : E_1



el significando con menor exponente se desplaza a la derecha para alinear

Ajuste

- ▶ Si $M \geq 2$, desplazar M a la derecha, incrementar E
- ▶ Si $M < 1$, desplazar M a la izquierda, decrementar E
- ▶ Si E fuera de rango, *overflow*
- ▶ Redondear M para que concuerde con la precisión de **frac**

- ▶ Comparando con un grupo abeliano
 - ¿Cerrado bajo la suma? – **Sí**
 - ¿Es conmutativa? – **Sí**
 - ¿Es asociativa? – **No**
 - Puede ocurrir un *overflow*; es inexacto (redondea)
 - Ej.:
 - ▶ $(3.14 + 1e10) - 1e10 = 0.0$
 - ▶ $3.14 + (1e10 - 1e10) = 3.14$
 - ¿Es 0 la identidad aditiva? – **Sí**
 - ¿Todo elemento tiene un inverso? – **Casi**
 - Excepto los infinitos y NaNs
- ▶ Monotonicidad
 - $a \geq b \ \& \ c \geq 0 \implies a + c \geq b + c?$ – **Casi**
 - Salvo por infinitos y/o NaNs

- ▶ Comparando con un anillo conmutativo
 - ¿Cerrado bajo la multiplicación? – Sí
 - ¿Es conmutativa la multiplicación? – Sí
 - ¿Es asociativa la multiplicación? – No
 - Puede ocurrir un *overflow*; es inexacto (redondea)
 - Ej.:
 - ▶ $(1e20 * 1e20) * 1e-20 = \text{inf}$
 - ▶ $1e20 * (1e20 * 1e-20) = 1e20$
 - ¿Es 1 la identidad multiplicativa? – Sí
 - ¿Es distributiva sobre la suma? – No
 - Puede ocurrir un *overflow*; es inexacto (redondea)
 - Ej.:
 - ▶ $1e20 * (1e20 - 1e20) = 0.0$
 - ▶ $1e20 * 1e20 - 1e20 * 1e20 = \text{NaN}$
- ▶ Monotonicidad
 - $a \geq b \ \& \ c \geq 0 \implies a * c \geq b * c?$ – Casi
 - Salvo por infinitos y/o NaNs